Algorithms and Development of Programmes for Biological Chemistry

Agata M. Kilar

Research Group Bioinformatics and Computational Biology
Department of Theoretical Chemistry
Währingerstrasse 17, 1090 Vienna, Austria

Summer 2026

Robots Game: Introduction


Robots Game
Tournament
Build. Collaborate. Compete.
Team Bots • Strategy • Coding • Final Showdown
BLUE TEAM
01 BYTE 120
03 LOOP 95
05 PIXEL 70
285
YELLOW TEAM
02 ALGO 110
04 NEXUS 100
210
01
03
05
04
02
FINAL SHOWDOWN
WHO WILL BE CHAMPION?

Outline

Game Setup

- The game is played on a grid with empty fields and walls (#).
- Each robot starts at a random empty position.
- A gold pot ($) is placed somewhere on the map.
- The goal is to finish the game with the **largest amount of gold**.

Rules for movement

- Each robot sees only a local area around itself, but it knows where the gold pot is.
- Robots can move to the 8 neighboring cells, but cannot pass through walls.
- If a robot tries to move into a wall (or outside the board), it crashes and loses **health**.
- After a crash, all remaining moves for that round are cancelled.

Moves are executed in alternating steps across all robots, not one robot at a time.

Collision avoidance and opponent prediction matter.

Health affects ability to move

- Each robot has health, with a maximum of **100**.
- Health is lost when the robot crashes.
- Crashing into a wall costs **25 health**.
- Crashing into another robot costs about **15–20 health**.
- At the beginning of each round, the robot heals by **10 health**, up to the maximum.
- If health drops below **30**, the robot is too weak to move.

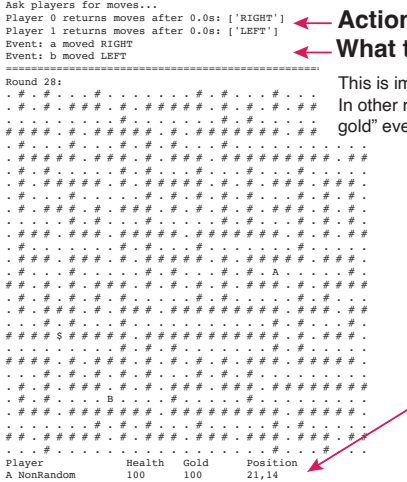
Gold

- At the beginning of each round, every robot gets **+1 gold**.
- A robot gains the gold from the pot by **moving onto the pot field**.
- The gold moves, and by default it relocates every 20 rounds unless someone takes it earlier.

Extras: buying more moves

- A player returns a **list of moves**, so in principle it may try several moves in one round.
- There is **no fixed hard limit** on the number of moves per round.
- **... but!** Each next moves becomes more expensive...
- If a player makes k moves in one round, the total cost is $1 + 2 + \cdots + k$.
- Example: 1 move costs 1 gold, 2 moves cost 3 gold, 3 moves cost 6 gold... etc.

Example Snapshot



Actions chosen by bots for this round

What the simulator actually executed

This is important because a chosen move is not always successful. In other rounds you may see crashes, cancelled moves, or "out of gold" events

. - empty field
 # - wall
$ - the gold pot

A - player A (NonRandom)
B - player B (SillyScout)

Stats of each player:
Health, Gold and current position on the map

The gold pot is at (4,9)
It currently contains 116 gold
12 rounds remain before relocation/timeout

Outline

Default Players

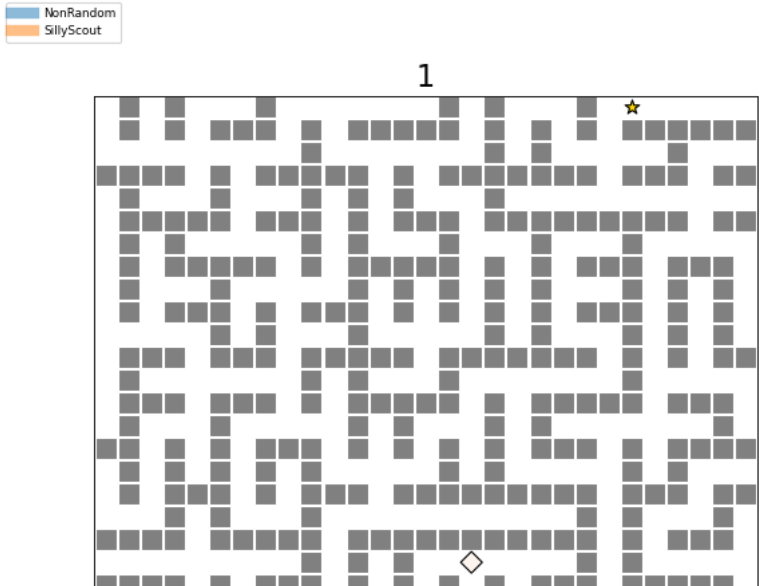
- **NonRandom** (`MyNonRandomPlayer`, from `test-RobotRace.py`)

  A simple greedy bot. It remembers discovered walls and makes one move per round toward the gold pot by choosing the neighboring field that looks closest to the goal.

- **SillyScout** (`MyPathFindingPlayer`, from `beatme-RobotRace.py`)

  A stronger pathfinding bot. It builds an internal map and uses shortest paths toward the gold pot, so it usually handles maze-like maps better than greedy movement.

Demo - Example Race



Outline

It's all about GitHub

As much as you are excited about tournament ;),
let's not forget we want to learn using GitHub

Project Organization

- You will work in **teams of 2–3 students**.
- Each team will design, implement, test, and improve its own robot player.
- Before the final tournament, we will have **three test rounds**.
- Each round is a chance to submit an improved version of your bot and compare it with the others.

Planned rounds:

- Intoduction: 5th May
- Round I: 12th May
- Round II: 2nd June
- Round III: 9th June
- Final tournament: 16th June

Working Together on GitHub

- Your team works in its own **fork** of the class repository.
- The official class repository is the place where the round tournament is run.
- One teammate creates the fork and adds the others as collaborators.
- The team develops and reviews changes in the fork.
- For each round, the final team submission is a **pull request** into the official round branch.

Working Together on GitHub

Team fork for collaboration, official round branch for submission

Official repo

Official class repository

official branches,
tournament runs here

Team workspace

Team fork

team members collaborate here

branches • commits • internal PRs

Round submission

Round submission

final PR into the
official round branch

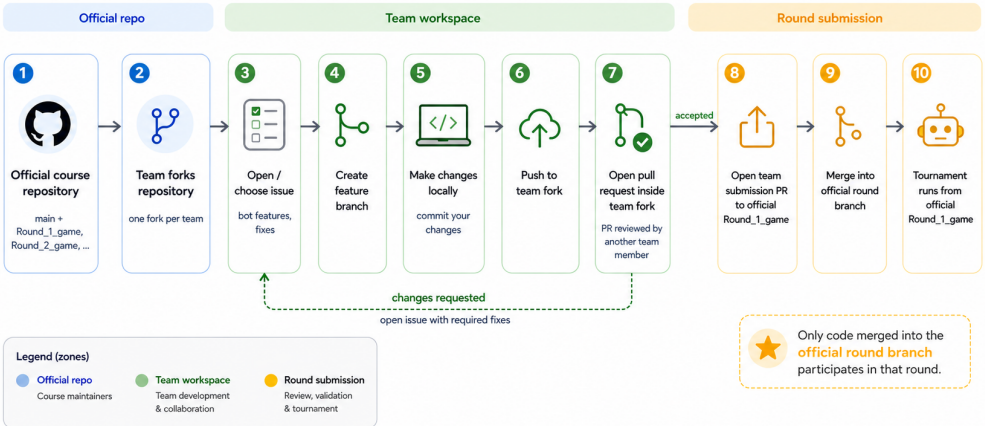
Only code merged into the **official round branch** participates in that round.

Expected Team Workflow

- I expect **every team member** to contribute through GitHub.
- Each student should create at least **one issue** describing a task, idea, bug, or improvement.
- Each student should work on that task in **their own branch** inside the team fork.
- When the work is ready, the student should open a **pull request** into the team branch for the current round.
- Another team member should then **review** the pull request and either:
  - accept and merge it, or
  - request changes / reject it.
- This way, your GitHub history will show both **individual contributions** and **team collaboration**.

Robot Game Team Workflow

One fork per team, one official branch per round



One-Time Setup for Each Team

1. One team member forks the official class repository.
2. That student invites the other team members as collaborators.
3. Everyone clones the team fork.
4. Everyone adds the official repository as `upstream`.

Repository Structure

- **Official repository**: game engine, official round branches, tournament code.
- **Team fork**: your private workshop for implementing and testing the bot.
- **origin**: your team fork.
- **upstream**: the official class repository.

Main rule: the tournament is run only from the official repository, not from your local machine and not directly from your fork.

Official Repository vs Team Fork

Official class repository

- contains the game engine
- contains official round branches
- is used to run the tournament
- receives team submissions as PRs

Team fork

- one fork per team
- one student owns the fork
- teammates are collaborators
- development and review happen here

Minimum GitHub Evidence for Each Round

Each team should have:

- at least one issue or task list for the round
- commits from more than one team member, when possible
- at least one internal PR or reviewed change
- one official round submission PR
- updated `README.md` or `strategy.md`
- a bot that can be loaded by the tournament script

Happy Hacking!